```
# -*- coding: utf-8 -*-
"""granite-3.3-2b-instruct.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/#fileId=https%3A//
huggingface.co/ibm-granite/granite-3.3-2b-instruct.ipynb
"""

!pip install -U transformers

"""## Local Inference on GPU
Model page: https://huggingface.co/ibm-granite/granite-3.3-2b-
instruct

⚠️ If the generated code snippets do not work, please open an
issue on either the [model repo](https://huggingface.co/ibm-
granite/granite-3.3-2b-instruct)
          and/or on [huggingface.js](https://github.com/
huggingface/huggingface.js/blob/main/packages/tasks/src/model-
libraries-snippets.ts) 🙏
"""

# Use a pipeline as a high-level helper
from transformers import pipeline

pipe = pipeline("text-generation", model="ibm-granite/
granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
pipe(messages)

# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/
granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained("ibm-granite/
granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
```

```
   messages,
   add_generation_prompt=True,
   tokenize=True,
   return_dict=True,
   return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)
print(tokenizer.decode(outputs[0]
[inputs["input_ids"].shape[-1]:]))

!pip install -q gradio transformers torch black autopep8
reportlab requests

!pip install -q gradio transformers torch black autopep8
reportlab requests ast-super-parser

import gradio as gr
from datetime import datetime
from pathlib import Path
import json
import re
import ast
from reportlab.lib.pagesizes import letter, A4
from reportlab.platypus import SimpleDocTemplate, Paragraph,
Spacer, PageBreak, Table, TableStyle, Preformatted
from reportlab.lib.styles import getSampleStyleSheet,
ParagraphStyle
from reportlab.lib.units import inch
from reportlab.lib import colors
import requests
import urllib.parse
import inspect

#
==================================================================
=============
# DOC-GENIE CORE LOGIC - IMPROVED VERSION
#
==================================================================
=============

class DocGenieAnalyzer:
    """Analyze Python functions and generate professional
docstrings"""
```

```
    @staticmethod
    def extract_function_signature(code):
        """Extract function name, parameters, return type from
code"""
        try:
            tree = ast.parse(code)
            func_def = tree.body[0]
            if not isinstance(func_def, ast.FunctionDef):
                return None, None

            func_name = func_def.name
            params = []

            for arg in func_def.args.args:
                param_type = "Any"
                if arg.annotation:
                    param_type = ast.unparse(arg.annotation) if
hasattr(ast, 'unparse') else "Any"
                params.append({
                    'name': arg.arg,
                    'type': param_type,
                    'default': None
                })

            for i, default in
enumerate(func_def.args.defaults):
                idx = len(params) - len(func_def.args.defaults)
+ i
                if idx >= 0:
                    params[idx]['default'] =
ast.unparse(default) if hasattr(ast, 'unparse') else
str(default)

            return_type = "Any"
            if func_def.returns:
                return_type = ast.unparse(func_def.returns) if
hasattr(ast, 'unparse') else "Any"

            return {
                'name': func_name,
                'params': params,
                'return_type': return_type
            }, func_def
        except:
```

```
            return None, None

    @staticmethod
    def analyze_function_logic(func_def, code):
        """Deep analysis of function body to understand actual
behavior"""
        try:
            body = func_def.body
            analysis = {
                'has_loops': False,
                'has_conditions': False,
                'has_return': False,
                'loop_types': [],
                'condition_types': [],
                'operations': [],
                'return_types': [],
                'variables_used': set(),
                'function_calls': [],
                'description': ""
            }

            # Walk through AST
            for node in ast.walk(func_def):
                if isinstance(node, ast.For):
                    analysis['has_loops'] = True
                    analysis['loop_types'].append('for')
                elif isinstance(node, ast.While):
                    analysis['has_loops'] = True
                    analysis['loop_types'].append('while')
                elif isinstance(node, ast.If):
                    analysis['has_conditions'] = True
                    analysis['condition_types'].append('if')
                elif isinstance(node, ast.Return):
                    analysis['has_return'] = True
                    if node.value:

analysis['return_types'].append(ast.unparse(node.value) if
hasattr(ast, 'unparse') else 'value')
                elif isinstance(node, ast.BinOp):
                    op = node.op
                    if isinstance(op, ast.Mult):

analysis['operations'].append('multiplication')
                    elif isinstance(op, ast.Add):
```

```
analysis['operations'].append('addition')
                    elif isinstance(op, ast.Sub):

analysis['operations'].append('subtraction')
                    elif isinstance(op, ast.Div):

analysis['operations'].append('division')
                elif isinstance(node, ast.Call):
                    if isinstance(node.func, ast.Name):

analysis['function_calls'].append(node.func.id)
                elif isinstance(node, ast.Name):
                    analysis['variables_used'].add(node.id)

            # Build detailed description
            description_parts = []

            if func_def.name:
                description_parts.append(f"Executes
{func_def.name} operation")

            if analysis['has_conditions']:
                description_parts.append("with conditional
logic")

            if analysis['has_loops']:
                description_parts.append("and iterates through
values")

            if 'multiplication' in analysis['operations'] and
'subtraction' in analysis['operations']:
                description_parts.append("performing
mathematical calculations")
            elif 'multiplication' in analysis['operations']:
                description_parts.append("using multiplication
operations")
            elif 'addition' in analysis['operations']:
                description_parts.append("using addition
operations")
            elif 'division' in analysis['operations']:
                description_parts.append("using division
operations")

            if analysis['return_types']:
                description_parts.append(f"and returns the
```

```
computed result")

            analysis['description'] = "
".join(description_parts) + "."
            if len(analysis['description']) < 20:
                analysis['description'] = "Performs computation
based on input parameters."

            return analysis
        except Exception as e:
            return {
                'has_loops': False,
                'has_conditions': False,
                'has_return': False,
                'description': "Executes function logic based
on provided parameters."
            }

    @staticmethod
    def generate_google_docstring(signature, analysis, code):
        """Generate accurate Google-style docstring"""
        summary = analysis.get('description', 'Processes input
and returns result.')

        docstring = f'    """{summary}\n'

        if signature['params']:
            docstring += '\n    Args:\n'
            for param in signature['params']:
                default_str = f" (default: {param['default']})"
if param['default'] else ""
                docstring += f"        {param['name']}
({param['type']}): Parameter for controlling {param['name']}
behavior{default_str}.\n"

        docstring += f"\n    Returns:\n
{signature['return_type']}: The result of the computation.\n"
        docstring += '    """\n'

        return docstring

    @staticmethod
    def generate_numpy_docstring(signature, analysis, code):
        """Generate accurate NumPy-style docstring"""
        summary = analysis.get('description', 'Processes input
```

```
and returns result.')

        docstring = f'    """{summary}\n'

        if signature['params']:
            docstring += '\n    Parameters\n    ----------\n'
            for param in signature['params']:
                default_str = f", default: {param['default']}"
if param['default'] else ""
                docstring += f"    {param['name']} :
{param['type']}{default_str}\n"
                docstring += f"        Parameter for
controlling {param['name']} behavior.\n"

        docstring += f"\n    Returns\n    -------\n
{signature['return_type']}\n"
        docstring += f"        The result of the computation.
\n"
        docstring += '    """\n'

        return docstring

analyzer = DocGenieAnalyzer()
generation_history = []

def generate_docstring(code, style="google"):
    """Generate docstring for Python function with accurate
analysis"""

    if not code.strip():
        return None, "❌ Empty code provided!"

    try:
        signature, func_def =
analyzer.extract_function_signature(code)
        if not signature:
            return None, "❌ Invalid Python function code!"

        # Deep analysis of the function
        analysis = analyzer.analyze_function_logic(func_def,
code)

        if style.lower() == "google":
            docstring =
analyzer.generate_google_docstring(signature, analysis, code)
```

```
        else:
            docstring =
analyzer.generate_numpy_docstring(signature, analysis, code)

        # Extract function definition line
        code_lines = code.split('\n')
        func_line = code_lines[0]

        # Build final function with docstring
        final_code = func_line + '\n' + docstring
        for line in code_lines[1:]:
            final_code += line + '\n'

        generation_history.append({
            'code': code,
            'docstring': docstring,
            'style': style,
            'timestamp': datetime.now().isoformat(),
            'analysis': analysis
        })

        return final_code, f"✅ {style.upper()} Docstring
Generated!"

    except Exception as e:
        return None, f"❌ Error: {str(e)}"

def process_upload(file):
    """Process uploaded Python file"""
    if not file:
        return None, "❌ No file uploaded!"

    try:
        content = file.read().decode('utf-8')
        return content, "✅ File loaded successfully!"
    except:
        return None, "❌ Failed to read file!"

def clear_history():
    """Clear generation history"""
    global generation_history
    generation_history = []
    return "✅ History cleared!"

def download_pdf():
```

```
    """Download all generated docstrings as PDF"""
    if not generation_history:
        return "❌ No documentation generated!", None

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"doc_genie_{timestamp}.pdf"

    try:
        doc = SimpleDocTemplate(filename, pagesize=A4,
topMargin=0.5*inch, bottomMargin=0.5*inch)
        styles = getSampleStyleSheet()
        story = []

        title_style = ParagraphStyle(
            'Title',
            parent=styles['Heading1'],
            fontSize=24,
            textColor='#FF6B6B',
            spaceAfter=20,
            alignment=1
        )

        code_style = ParagraphStyle(
            'Code',
            parent=styles['Normal'],
            fontSize=9,
            fontName='Courier',
            backColor='#F0F0F0',
            borderColor='#CCCCCC',
            borderWidth=1,
            borderPadding=6
        )

        story.append(Paragraph("📚 Doc-Genie Documentation
Report", title_style))
        story.append(Paragraph(f"Generated:
{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
styles['Normal']))
        story.append(Spacer(1, 0.3*inch))

        for i, gen in enumerate(generation_history, 1):
            story.append(Paragraph(f"<b>Documentation #{i}</
b>", styles['Heading2']))
            story.append(Paragraph(f"<b>Style:</b>
{gen['style'].upper()}", styles['Normal']))
```

```
            story.append(Spacer(1, 0.1*inch))

            story.append(Paragraph("<b>Original Code:</b>",
styles['Normal']))
            story.append(Preformatted(gen['code'], code_style))
            story.append(Spacer(1, 0.15*inch))

            story.append(Paragraph("<b>Generated Docstring:</
b>", styles['Normal']))
            story.append(Preformatted(gen['docstring'],
code_style))
            story.append(Spacer(1, 0.3*inch))

            if i % 2 == 0 and i != len(generation_history):
                story.append(PageBreak())

        story.append(Spacer(1, 0.3*inch))
        story.append(Paragraph(f"Total Documentations:
{len(generation_history)}", styles['Normal']))

        doc.build(story)
        return f"✅ PDF Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None

def download_txt():
    """Download all generated docstrings as TXT"""
    if not generation_history:
        return "❌ No documentation generated!", None

    content = f"Doc-Genie Documentation Report\nGenerated:
{datetime.now()}\n{'='*80}\n\n"

    for i, gen in enumerate(generation_history, 1):
        content += f"Documentation #{i}\n"
        content += f"Style: {gen['style'].upper()}\n"
        content += f"Generated: {gen['timestamp']}\n"
        content += f"{'-'*80}\n\n"
        content += f"ORIGINAL CODE:\n{gen['code']}\n\n"
        content += f"GENERATED DOCSTRING:\n{gen['docstring']}
\n\n"
        content += f"{'-'*80}\n\n"

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"doc_genie_{timestamp}.txt"
```

```
    try:
        Path(filename).write_text(content)
        return f"✅ TXT Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None

def generate_whatsapp_link():
    """Generate WhatsApp share link"""
    if not generation_history:
        return "❌ No documentation to share!"

    message = "📚 *Doc-Genie Documentation*\n\n"
    for i, gen in enumerate(generation_history[-3:], 1):
        message += f"*Doc {i}:*\n"
        message += f"Style: {gen['style']}\n"
        message += f"`{gen['code'][:60]}...`\n\n"

    encoded = urllib.parse.quote(message)
    link = f"https://wa.me/?text={encoded}"

    return f"✅ WhatsApp Link:\n\n{link}\n\n📱 Share with
team!"

def generate_facebook_link():
    """Generate Facebook share link"""
    if not generation_history:
        return "❌ No documentation to share!"

    message = f"I generated {len(generation_history)}
professional Python function docstrings with Doc-Genie! Check
it out!"
    encoded = urllib.parse.quote(message)
    link = f"https://www.facebook.com/sharer/sharer.php?
quote={encoded}"

    return f"✅ Facebook Link:\n\n{link}\n\n👍 Share on
Facebook!"

print("🚀 Loading Doc-Genie Improved Version...")

with gr.Blocks(
    title="Doc-Genie - Python Documentation Generator",
    theme=gr.themes.Soft(),
    css="""
```

```
    body { background: linear-gradient(135deg, #1a1a2e 0%,
#16213e 100%); }
    .gradio-container { background: linear-gradient(135deg,
#1a1a2e 0%, #16213e 100%); }
    .button-primary { background: linear-gradient(135deg,
#FF6B6B 0%, #FF8E72 100%); }
    .button-secondary { background: linear-gradient(135deg,
#4ECDC4 0%, #44A08D 100%); }
    .code-block { background: #0f3460; border: 2px solid
#4ECDC4; border-radius: 8px; padding: 15px; }
    """
) as demo:

    gr.Markdown("""
    # 📚 Doc-Genie v2.0
    ## Professional Python Function Docstring Generator

    Transform your Python functions with AI-powered, accurate
documentation!
    """)

    with gr.Row():
        with gr.Column(scale=3):
            # Code Input Area
            gr.Markdown("### 📝 Input Function Code")

            code_input = gr.Code(
                language="python",
                label="Python Function",
                lines=10,
                value="def calculate_factorial(n):\n    if n ==
0:\n        return 1\n    return n * calculate_factorial(n-1)"
            )

            # File Upload
            file_upload = gr.File(label="📤 Or Upload .py
File", file_types=['.py'])
            upload_status = gr.Textbox(label="Upload Status",
interactive=False, lines=1)

            def process_and_display(file):
                if file:
                    content, status = process_upload(file)
                    return content, status
                return "", "No file"
```

```
            file_upload.change(process_and_display,
inputs=[file_upload], outputs=[code_input, upload_status])

            # Style Selection
            with gr.Row():
                docstring_style = gr.Radio(["google", "numpy"],
value="google", label="🎨 Docstring Style")
                generate_btn = gr.Button("✨ Generate
Docstring", size="lg", variant="primary", scale=2)

            # Output Area
            gr.Markdown("### 📄 Generated Output")

            code_output = gr.Code(
                language="python",
                label="Function with Docstring",
                lines=15,
                interactive=False
            )

            gen_status = gr.Textbox(label="Status",
interactive=False, lines=1)

            generate_btn.click(
                generate_docstring,
                inputs=[code_input, docstring_style],
                outputs=[code_output, gen_status]
            )

        with gr.Column(scale=1):
            gr.Markdown("### 🎮 Controls")

            clear_btn = gr.Button("🗑️ Clear History",
variant="stop", size="sm")
            clear_btn.click(clear_history,
outputs=[upload_status])

            gr.Markdown("---")
            gr.Markdown("### 📥 Export")

            pdf_btn = gr.Button("📄 Download PDF", size="sm",
variant="primary")
            txt_btn = gr.Button("📋 Download TXT", size="sm",
variant="primary")
```

```
            pdf_file = gr.File(label="PDF File", visible=True)
            txt_file = gr.File(label="TXT File", visible=True)

            export_status = gr.Textbox(label="Export Status",
interactive=False, lines=2)

            pdf_btn.click(download_pdf, outputs=[export_status,
pdf_file])
            txt_btn.click(download_txt, outputs=[export_status,
txt_file])

            gr.Markdown("---")
            gr.Markdown("### 📱 Share")

            whatsapp_btn = gr.Button("💬 WhatsApp", size="sm",
variant="secondary")
            facebook_btn = gr.Button("👍 Facebook", size="sm",
variant="secondary")

            share_output = gr.Textbox(label="Share Link",
interactive=False, lines=4)

            whatsapp_btn.click(generate_whatsapp_link,
outputs=[share_output])
            facebook_btn.click(generate_facebook_link,
outputs=[share_output])

            gr.Markdown("---")
            gr.Markdown("""
            ### 📊 Features

            **Improvements:**
            - ✅ Accurate logic analysis
            - ✅ Contextual descriptions
            - ✅ Operation detection
            - ✅ Loop & condition recognition
            - ✅ Better PDF export
            - ✅ Consistent output

            ### 🎯 Supported Styles

            **Google:** Standard format
            **NumPy:** Scientific format
            """)
```

```
print("\n✅ Doc-Genie v2.0 Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```